

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I JAYADEEP.J(192511304) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

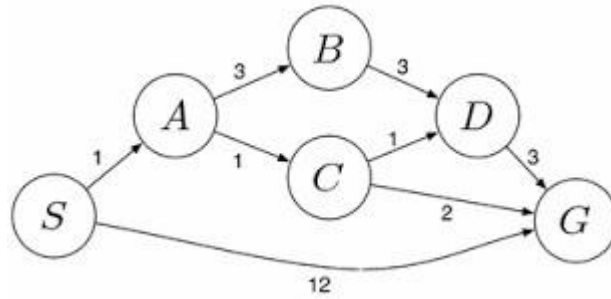
Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

ANSWER SHEET

Artificial Intelligence (CSA17) – CO1

Question 1: Water Jug Problem

The problem is to measure exactly 2 gallons in the 4-gallon jug, using only a 4-gallon jug, a 3-gallon jug and a pump. Neither jug has markings, so the only way to know how much water is in a jug is to fill it completely, empty it completely, or pour until one jug is full or the other is empty.

Problem formulation

- **State:** an ordered pair (x, y) , where x is the water in the 4-gallon jug ($0 \leq x \leq 4$) and y is the water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** $(0, 0)$, both jugs empty.
- **Goal state:** $(2, n)$ for any n , since only the 4-gallon jug has to hold exactly 2 gallons.

Operators (production rules)

No.	Rule	Meaning
1	$(x, y), x < 4 \rightarrow (4, y)$	Fill the 4-gallon jug from the pump
2	$(x, y), y < 3 \rightarrow (x, 3)$	Fill the 3-gallon jug from the pump
3	$(x, y), x > 0 \rightarrow (0, y)$	Empty the 4-gallon jug on the ground
4	$(x, y), y > 0 \rightarrow (x, 0)$	Empty the 3-gallon jug on the ground
5	$(x, y), x + y \geq 4, y > 0 \rightarrow (4, y - (4 - x))$	Pour from the 3-gallon jug into the 4-gallon jug until it is full
6	$(x, y), x + y \geq 3, x > 0 \rightarrow (x - (3 - y), 3)$	Pour from the 4-gallon jug into the 3-gallon jug until it is full
7	$(x, y), x + y \leq 4, y > 0 \rightarrow (x + y, 0)$	Pour all of the 3-gallon jug into the 4-gallon jug
8	$(x, y), x + y \leq 3, x > 0 \rightarrow (0, x + y)$	Pour all of the 4-gallon jug into the 3-gallon jug

Solution

Searching breadth first from $(0, 0)$ gives a shortest solution of six moves. The route I followed is shown below.

Step	4-gal jug (x)	3-gal jug (y)	Action / rule used
0	0	0	Start
1	4	0	Fill the 4-gallon jug (rule 1)
2	1	3	Pour from the 4-gallon into the 3-gallon until the 3-gallon is full (rule 6); 1 gallon is left behind
3	1	0	Empty the 3-gallon jug (rule 4)

Step	4-gal jug (x)	3-gal jug (y)	Action / rule used
4	0	1	Pour the 1 gallon from the 4-gallon into the 3-gallon (rule 8)
5	4	1	Fill the 4-gallon jug again (rule 1)
6	2	3	Pour from the 4-gallon into the 3-gallon until it is full (rule 6); it only needs 2 more gallons, so exactly 2 stay in the 4-gallon jug

State sequence: $(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3)$. The 4-gallon jug now holds exactly **2 gallons**, so the goal is reached.

There is a second route that is just as short: $(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2)$. This also ends with 2 gallons, but in the 3-gallon jug, so one extra pour would be needed to move it. That is why I preferred the first route for this particular goal.

Question 2: Mars Rover Agent

A good way to organise this is the PEAS description (Performance, Environment, Actuators, Sensors), since each part of the question matches one of these.

i. Percepts

The percepts are whatever the rover's sensors deliver at each moment. For a rover exploring Mars these would be:

- camera images (colour, stereo) of the terrain, rocks and soil, used to spot obstacles and interesting samples;
- proximity, lidar or hazard sensors that detect rocks, slopes and craters close by;
- wheel odometry, gyroscope and accelerometer readings, which tell the rover its position, tilt and speed;
- spectrometer and chemical/mineral sensor data from analysing a sample (for example water, carbon or iron content);
- temperature, dust and radiation readings, plus the solar panel or battery level;
- commands and signals received from Earth, and the time of day (Mars sol).

ii. Operating environment

The environment is the Martian surface. Characterising it using the standard properties:

Property	Type	Reason
Observability	Partially observable	The rover only sees what is within range of its sensors; the terrain behind a hill or under dust is hidden.
Agents	Single agent	Only one rover is acting on the area. Other things (wind, dust storms) are natural events, not competing agents.

Property	Type	Reason
Determinism	Stochastic	Wheels may slip in loose sand, and sensor readings are noisy, so the result of an action is not fully predictable.
Episodic / sequential	Sequential	Current moves affect later ones, e.g. battery used now limits what can be done later, and a bad move can trap the rover.
Static / dynamic	Dynamic	Dust storms, changing light and temperature change while the rover is still deciding.
Discrete / continuous	Continuous	Position, speed, temperature and so on take continuous values.
Known / unknown	Largely unknown	Much of the terrain has not been mapped, so the rover has to learn it as it goes.

iii. Actions

The actions are carried out by the actuators:

- move forward or backward, turn left or right, change speed, stop, climb or avoid an obstacle;
- steer or rotate the camera mast and point the antenna towards Earth;
- extend the robotic arm, drill, scoop or grind to collect a sample;
- place the sample into the onboard instruments, run the analysis, and store the data;
- transmit data and images to Earth, receive instructions;
- manage energy, e.g. recharge using solar panels, go into sleep or safe mode, heat the electronics during cold nights.

iv. Evaluating performance

Performance is judged against the mission goals, not against what the rover does for its own sake. Sensible measures would be:

- number and scientific value of the samples collected and correctly analysed;
- area of terrain explored, or the number of target sites reached;
- safety, meaning no collisions, tip-overs or getting stuck (a lost rover ends the mission);
- energy and time efficiency, i.e. how much was achieved per unit of battery and per sol;
- accuracy and amount of data successfully sent back to Earth;
- how long the rover stays operational (mission lifetime).

In practice the measure would be a weighted combination of these, with safety given the highest weight, because the rover cannot be repaired.

v. Most suitable agent architecture

I think the best choice is a **hybrid, model-based agent that is goal-based and utility-based, with a learning element**. Reasons:

- **Model-based**, because the environment is only partly observable. The rover has to keep an internal map and a record of where it has been, what is blocked and what its battery state is.
- **Goal-based**, because its job is to reach particular places and collect specific samples, which needs planning and search. A simple reflex agent cannot plan a route around a crater.
- **Utility-based**, because many goals compete: a sample may be valuable but the route is risky or uses too much energy. A utility function lets it trade off value, risk and power.
- **Learning**, since Mars is unknown. The rover improves its terrain classification and navigation from experience.

A thin reflex layer sits underneath for emergencies, such as stopping immediately when the tilt sensor exceeds a safe angle. This is needed because a signal from Earth takes several minutes, so the rover cannot wait to be told what to do.

Question 3: 8-Queens Problem

The aim is to place eight queens on an 8×8 board so that no two share a row, column or diagonal. The configuration in the figure is not a solution because the queen in the first column and the queen in the last column are on the same diagonal, which is a good reminder that all three kinds of attack need checking.

Problem components (incremental formulation)

- **States**: any arrangement of 0 to 8 queens on the board with no queen attacking another. Placing the queens one column at a time, the state space shrinks to about 2,057 states.
- **Initial state**: an empty board with no queens.
- **Actions**: put a queen in the leftmost empty column, in a row where it is not attacked by any queen already placed.
- **Transition model**: returns the board with the new queen added.
- **Goal test**: 8 queens are on the board and none are attacked.
- **Path cost**: not important here; only the final arrangement matters, so every step can simply cost 1.

The formulation matters a lot for the size of the search space. If the problem is stated naively (place any queen on any square, with no checking), there are $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ possible sequences. Putting one queen in each column gives $8^8 \approx 16.7$ million complete states. Only allowing safe placements, as above, cuts this to 2,057, which is why the formulation is chosen carefully.

Exploring the search space

Since a solution is always found at depth 8 (one queen per column) and the path to it is irrelevant, depth-first search with backtracking suits this problem. For example, we place a queen at column 1 row 1, then column 2 row 3 (the first row not attacked), then column 3 row 5, and so on. When a column has no safe row left, the search backtracks to the previous column and tries the next row. Breadth-first search would also work but needs to store a huge frontier, since all the partial boards

at one depth must be kept. Hill climbing with the number of attacking pairs as the heuristic is another option for the complete-state version, but it can get stuck at local minima.

There are 92 solutions in total, but only 12 are distinct once rotations and reflections are ignored.

Question 4: OLA Cab Booking

The suitable agent is a **goal-based problem-solving agent**, extended with a utility function. The goal is clear: reach the destination from the pickup location. But the customer can pick between mini, micro, sedan, shared, prime and so on, so the agent has to compare several ways of achieving the goal using fare, comfort, waiting time and travel time. That comparison is what a utility-based element does. The agent formulates the goal, formulates the problem, searches for a solution (a ride plan) and then executes it.

Problem formulation for this app:

- Initial state: customer at the pickup location, no cab booked.
- Actions: choose cab type, request the ride, confirm driver, travel along a route.
- Goal test: customer is at the destination.
- Path cost / utility: a weighted mix of fare, estimated time, waiting time and comfort preference.

Pseudocode

```
function CAB-BOOKING-AGENT(percept) returns an action
  persistent: seq, an action sequence (initially empty)
               state, the current description of the world
               goal, problem, initially null

  state <- UPDATE-STATE(state, percept)    // location, traffic, available cabs, fares
  if seq is empty then
    goal <- FORMULATE-GOAL(state)          // reach destination D from pickup P
    problem <- FORMULATE-PROBLEM(state, goal)
    seq <- SEARCH(problem)
    if seq = failure then return a null action (no cab available)
  action <- FIRST(seq)
  seq <- REST(seq)
  return action

function SEARCH(problem) returns a booking plan
  options <- list of cab types available near pickup point P
  best <- null; bestUtility <- -infinity
  for each cab in options do
    fare <- ESTIMATE-FARE(cab, P, D)
    eta <- ESTIMATE-WAITING-TIME(cab, P)
    time <- ESTIMATE-TRAVEL-TIME(cab, P, D)
    u <- UTILITY(fare, eta, time, comfort(cab), customer-preference)
    if u > bestUtility then best <- cab; bestUtility <- u
```

```

if best = null then return failure
route <- SHORTEST-PATH(P, D)           // e.g. uniform cost / A* on the road map
return [SELECT(best), REQUEST-RIDE(best), CONFIRM-DRIVER, FOLLOW(route), END-TRIP]

```

In this version the customer's own selection can replace the utility step: if the customer chooses a specific cab type, the agent just filters the options to that type and then books it. The rest of the structure stays the same.

Question 5: Uniform Cost Search on the Delivery Network

From the graph, the edges and their costs are: $S-A = 1$, $S-G = 12$, $A-B = 3$, $A-C = 1$, $B-D = 3$, $C-D = 1$, $C-G = 2$ and $D-G = 3$. UCS always expands the node in the frontier with the lowest path cost $g(n)$, which makes it complete and optimal for non-negative edge costs. The frontier is a priority queue ordered by g .

Step	Node expanded (g)	Nodes generated / updated	Frontier after step (node:g)
1	S (0)	$A = 1$, $G = 12$	A:1, G:12
2	A (1)	$B = 1+3 = 4$, $C = 1+1 = 2$	C:2, B:4, G:12
3	C (2)	$D = 2+1 = 3$, $G = 2+2 = 4$ (better than 12, so it replaces it)	D:3, B:4, G:4
4	D (3)	G via $D = 3+3 = 6$, which is worse than 4, so it is ignored	B:4, G:4
5	B (4)	D via $B = 4+3 = 7$, worse than 3, ignored	G:4
6	G (4)	Goal test succeeds, stop	—

The explored set is $\{S, A, C, D, B\}$, and the goal is found when G is popped from the queue (not when it is first generated, which is what keeps UCS optimal). Tracing back through the parents gives the path $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of 4.

For comparison, the other routes are $S \rightarrow G = 12$, $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 6$ and $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 10$, so the chosen path really is the cheapest. In practical terms, the company should send the packages along S, A, C, G , which costs 4 units of fuel plus time.